

Importing Libraries

```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like
competitions, datasets, and models to your session
# Learn more about kagglehub:
https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')

/kaggle/input/datasets/parvmodi/cgpa-vs-package-in-lpa/placement.csv
```

Importing Warnings:

```
import warnings
warnings.filterwarnings("ignore")
```

Loading the Data from the Path Given:

```
df=pd.read_csv("/kaggle/input/datasets/parvmodi/cgpa-vs-package-in-
lpa/placement.csv")
```

Checking the Dataset upto 5 Rows:

```
df.head(5)
```

	cgpa	package
0	6.89	3.26
1	5.12	1.98
2	7.82	3.25
3	7.42	3.67
4	6.94	3.57

Example: 01

Creating a 10 datapoints dataset for visualization.

```
df_10 = df[:10]
```

```
df_10
```

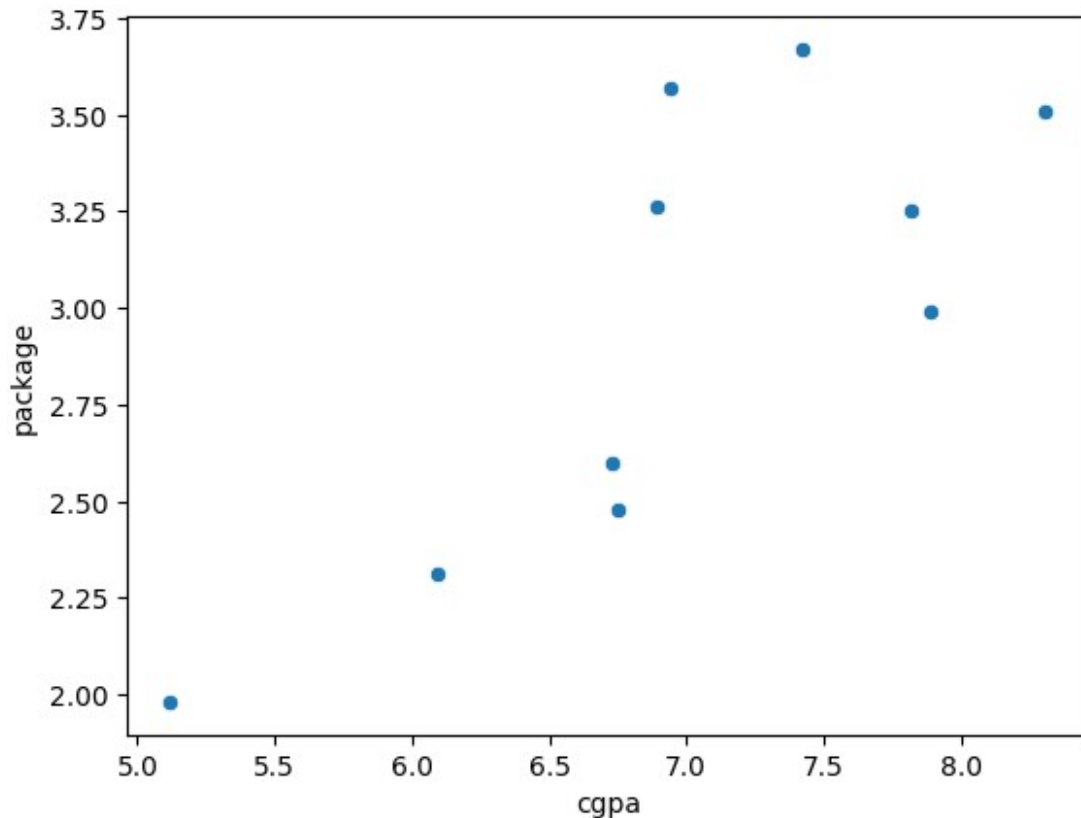
	cgpa	package
0	6.89	3.26
1	5.12	1.98
2	7.82	3.25
3	7.42	3.67
4	6.94	3.57
5	7.89	2.99
6	6.73	2.60
7	6.75	2.48
8	6.09	2.31
9	8.31	3.51

Imported Seaborn for Visualizing our Data:

```
import seaborn as sns
```

```
sns.scatterplot(data = df_10,  
                x = df_10["cgpa"],  
                y = df_10["package"]  
)
```

```
<Axes: xlabel='cgpa', ylabel='package'>
```



Lets create our Model

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()
```

Train our model on our Data:

```
model.fit(df_10[["cgpa"]],df_10["package"])  
LinearRegression()
```

Lets check the Parameters our model got after training:

Equation: $y = m \cdot x + c$

x -> independent variable (cgpa)

y -> dependent variable (package)

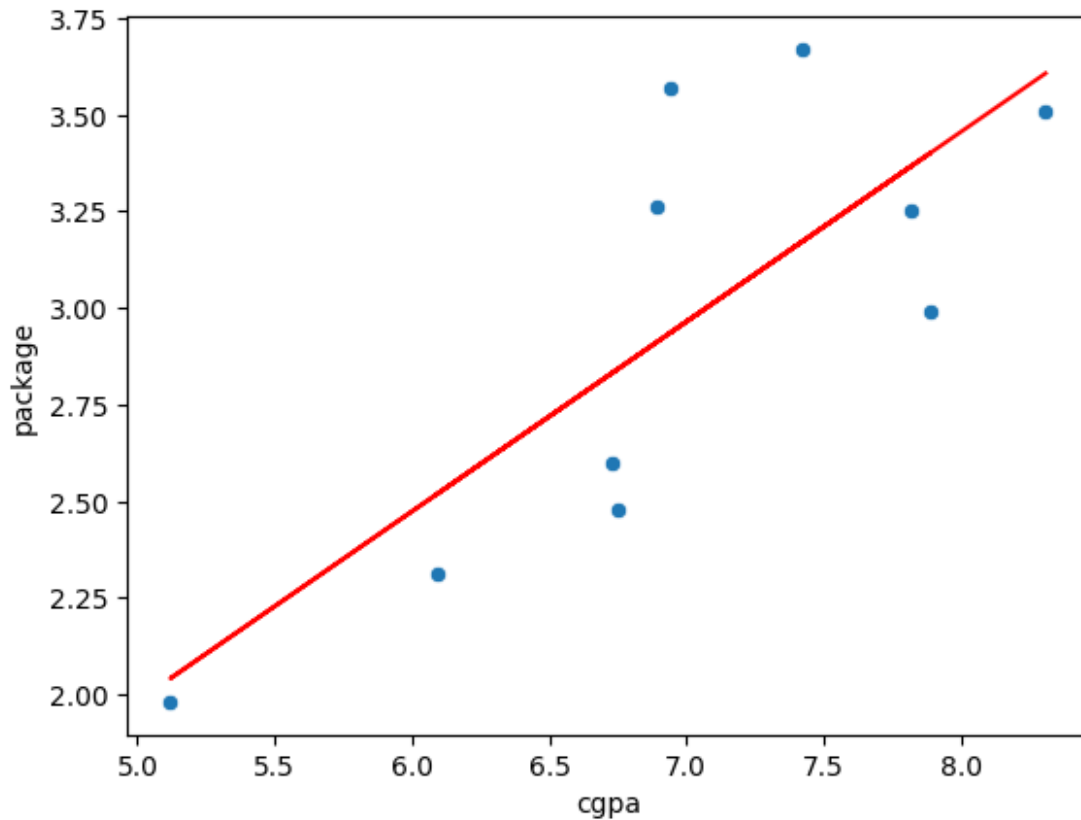
m -> slope/coefficient

c -> intercept

```
m = model.coef_  
print(f": Coef: {m}")  
  
: Coef: [0.49105006]  
  
c = model.intercept_  
print(f"Interercept: {c}")  
  
Interercept: -0.4733861893363027
```

Lets try to Visualize Best fit Line:

```
import matplotlib.pyplot as plt  
  
x = df_10["cgpa"]  
y = df_10["package"]  
  
y_pred = model.predict(df_10[["cgpa"]])  
sns.scatterplot(data = df_10, x = x, y = y)  
plt.plot(x, y_pred, color="red")  
[<matplotlib.lines.Line2D at 0x7b973a987110>]
```



Checking y:

```
print(y)
0    3.26
1    1.98
2    3.25
3    3.67
4    3.57
5    2.99
6    2.60
7    2.48
8    2.31
9    3.51
Name: package, dtype: float64
```

Lets analyse our model with evaluation metrics:

```
from sklearn.metrics import
mean_absolute_error, mean_squared_error, root_mean_squared_error, r2_score
```

1. Mean Absolute Error: (MAE)

Manual Calc.

```
mae = np.sum(np.abs((y - y_pred)))/len(y)
print(f"MAE calc Manully: {mae}")
MAE calc Manully: 0.29706897708387314
```

With Scikit-learn:

```
mae = mean_absolute_error(y,y_pred)
print(f"MAE with sklearn: {mae}")
MAE with sklearn: 0.29706897708387314
```

2. Mean Squared Error (MSE):

Manually:

```
mse = np.sum((y - y_pred)**2)/len(y)
print(f"MSE calc Manually: {mse}")
MSE calc Manually: 0.11987595659200753
```

With Scikit-learn:

```
mse = mean_squared_error(y,y_pred)
print(f"MAE with sklearn: {mse}")
MAE with sklearn: 0.11987595659200753
```

3. Root Mean Squared Error (RMSE):

Manually:

```
rmse = np.sqrt(mse)
print(f"RMSE manually: {rmse}")
RMSE manually: 0.3462310739838461
```

With Skicit-learn:

```
rmse = root_mean_squared_error(y,y_pred)
print(f"RMSE with sklearn :{rmse}")
RMSE with sklearn :0.3462310739838461
```

4.R2_Score:

Manually:

```
y_mean = np.mean(y)
sse_reg = np.sum((y - y_pred) ** 2)
sse_mean = np.sum((y - y_mean) ** 2)
r2_manual = 1 - (sse_reg / sse_mean)
print(f"R2_Score Manually: {r2_manual}")
R2_Score Manually: 0.6128737806081344
```

R2_Score with Scikit-learn:

```
r2_sklearn = r2_score(y,y_pred)
print(f"R2_Score with sklearn: {r2_sklearn}")
R2_Score with sklearn: 0.6128737806081344
```

Example: 02

We will create a linear data ourself.

```
data = {
    "X": [1,2,3,4,5,6,7],
    "y": [1.5,3.8,6.7,9.0,11.2,13.6,16]
}

print(f"Data: {data}")

Data: {'X': [1, 2, 3, 4, 5, 6, 7], 'y': [1.5, 3.8, 6.7, 9.0, 11.2, 13.6, 16]}
```

Convert our data into a pandas dataframe:

```
df = pd.DataFrame(data)
df
```

	X	y
0	1	1.5
1	2	3.8
2	3	6.7
3	4	9.0
4	5	11.2
5	6	13.6
6	7	16.0

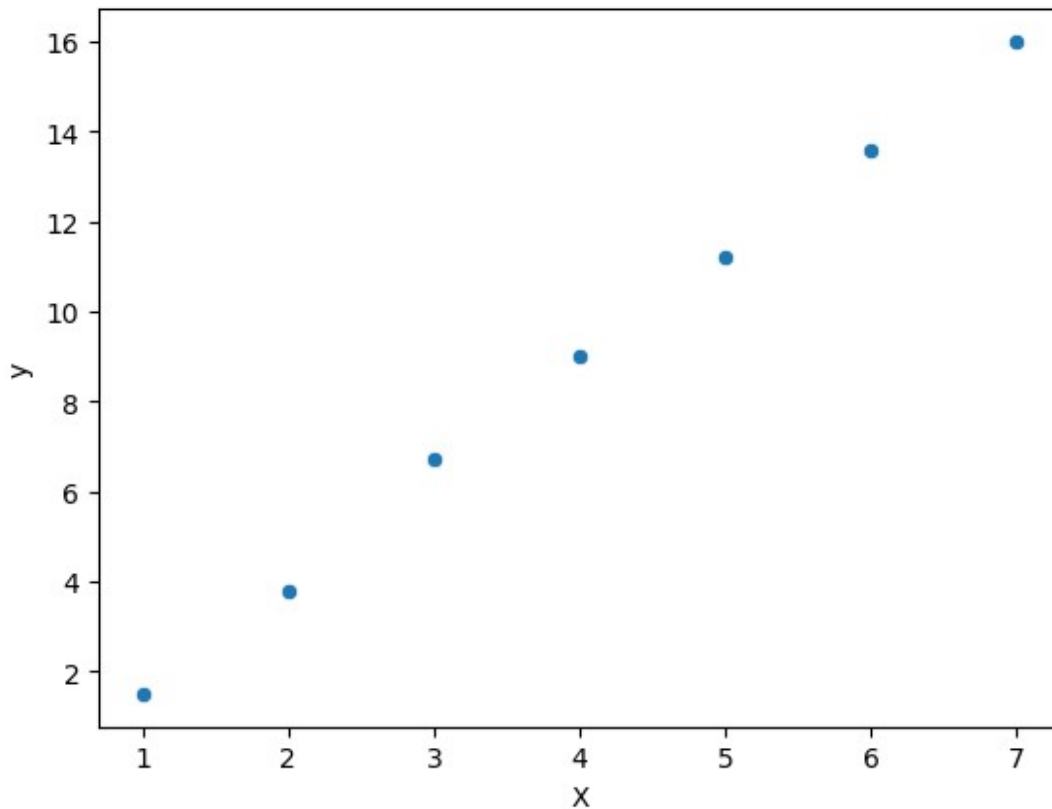
```
df.columns
```

```
Index(['X', 'y'], dtype='object')
```

Lets plot our data points:

```
sns.scatterplot(
    data = df,
    x = df["X"],
    y = df["y"]
)
```

```
<Axes: xlabel='X', ylabel='y'>
```

Lets Train our model:

```
model = LinearRegression()
model.fit(df[["X"]],df[["y"]])
LinearRegression()
X = df["X"]
y = df["y"]
y_pred = model.predict(df[["X"]])
```

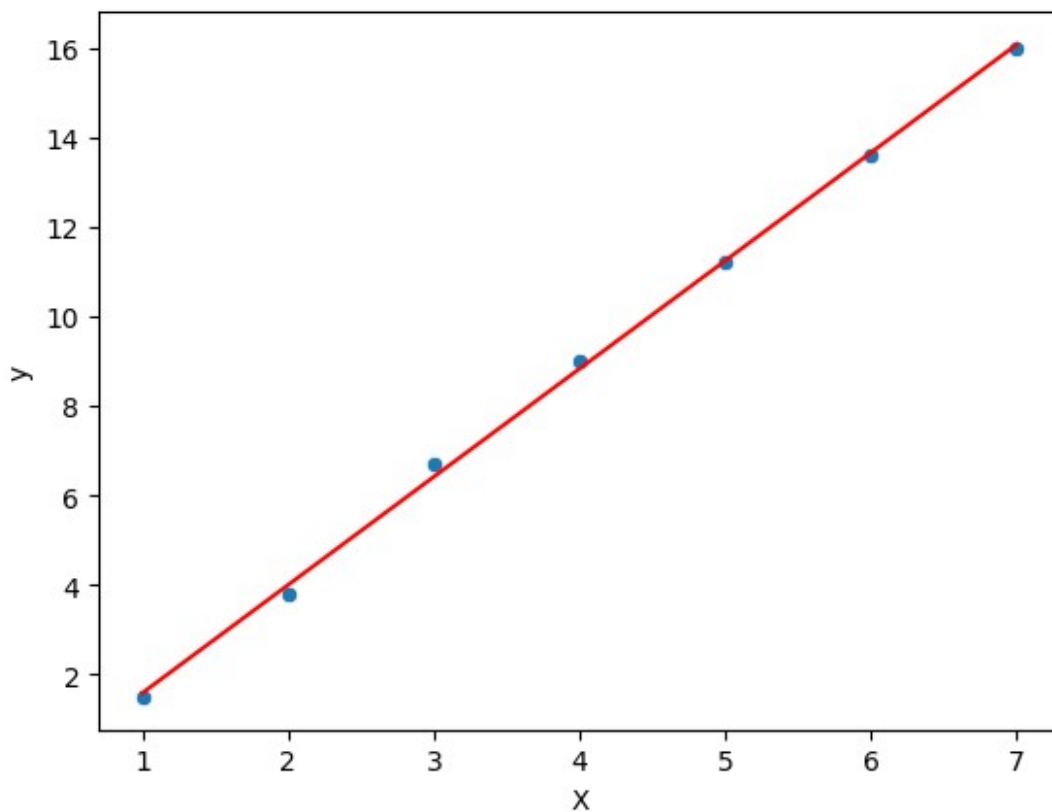
Testing our model with new data point:

```
given_cgpa = 4
predicted_package = model.predict(np.array([[given_cgpa]]))
print(f"Predicted Package: {predicted_package[0]}")
```

Predicted Package: 8.82857142857143

Lets try to draw line of best fit:

```
sns.scatterplot(  
    data=df,  
    x = df["X"],  
    y = df["y"]  
)  
  
plt.plot(  
    df["X"],  
    y_pred,  
    color="red"  
)  
[<matplotlib.lines.Line2D at 0x7b97371914f0>]
```



Lets Evaulate our model performance using metrics:

```
# 1. Mean Absolute Error (MAE)
mae = mean_absolute_error(y,y_pred)
print("Mean Absolute Error (MAE):", mae)

# 2. Mean Squared Error (MSE)
mse = mean_squared_error(y, y_pred)
print("Mean Squared Error (MSE):", mse)

# 3. Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)
print("Root Mean Squared Error (RMSE):", rmse)

# 4. R-squared (R2)
r2 = r2_score(y, y_pred)
print("R-squared (R2):", r2)

Mean Absolute Error (MAE): 0.13061224489795928
Mean Squared Error (MSE): 0.024081632653061243
Root Mean Squared Error (RMSE): 0.15518257844571742
R-squared (R2): 0.99896818873402
```

Lets work on our whole Data:

```
df=pd.read_csv("/kaggle/input/datasets/parvmodi/cgpa-vs-package-in-
lpa/placement.csv")

df.shape

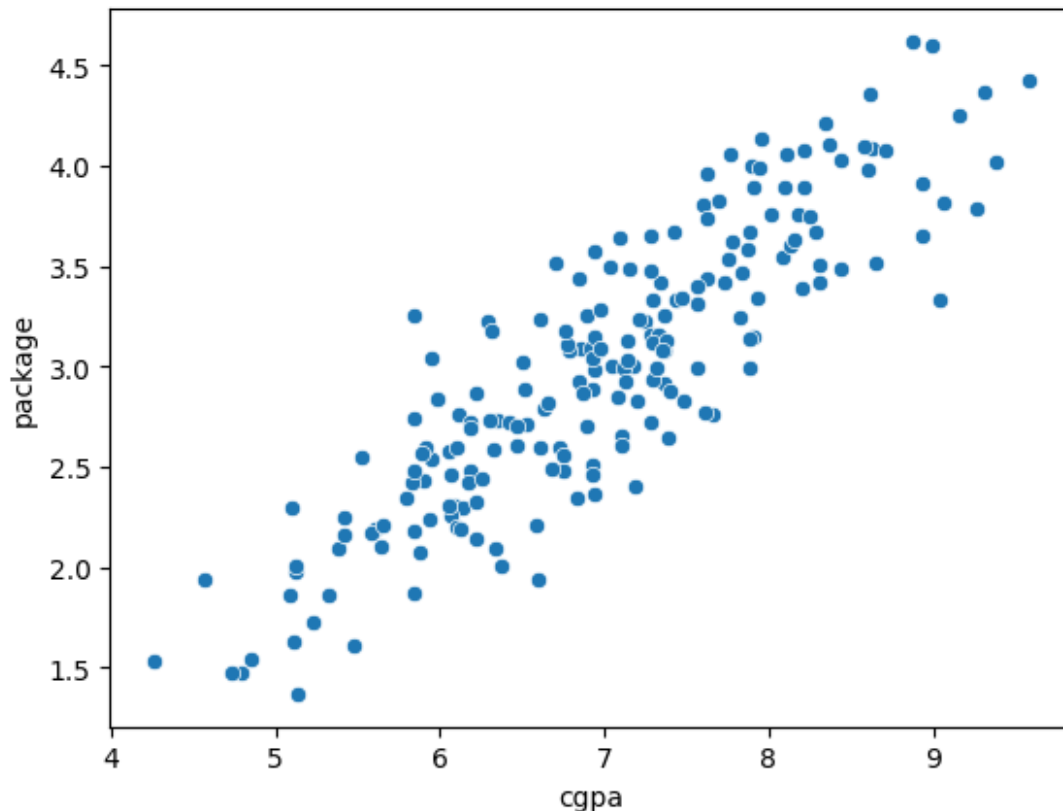
(200, 2)

df.columns

Index(['cgpa', 'package'], dtype='object')

sns.scatterplot(
    data = df,
    x = df["cgpa"],
    y = df["package"]
)

<Axes: xlabel='cgpa', ylabel='package'>
```



Lets do the Train Test Splitting of our Data:

```
from sklearn.model_selection import train_test_split

X = df[["cgpa"]]
y = df[["package"]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.20, random_state = 42)

print(f"X_train Shape: {X_train.shape}")
print(f"X_test Shape: {X_test.shape}")
print(f"y_train Shape: {y_train.shape}")
print(f"y_test Shape: {y_test.shape}")

X_train Shape: (160, 1)
X_test Shape: (40, 1)
y_train Shape: (160, 1)
y_test Shape: (40, 1)
```

Lets train our Model:

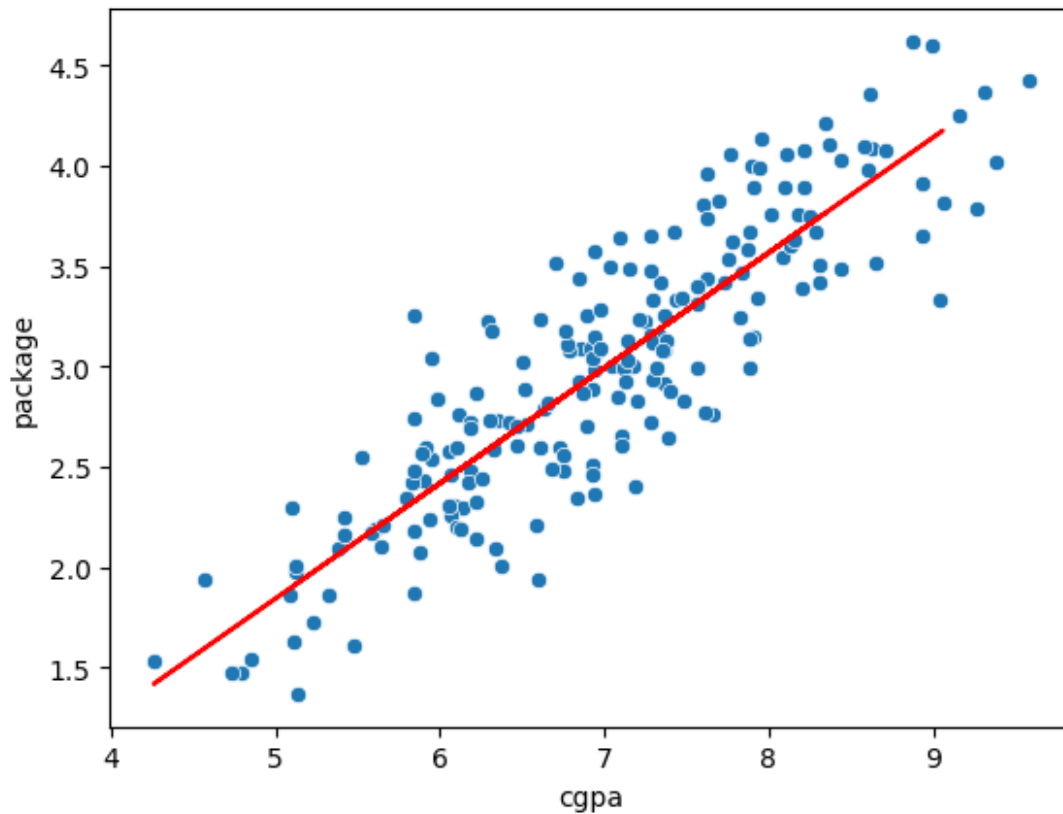
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train,y_train)
LinearRegression()
y_pred = model.predict(X_test)
```

Lets try to draw the Line of Best fit:

```
sns.scatterplot(
    data = df,
    x = df["cgpa"],
    y = df["package"]
)

plt.plot(
    X_test,
    y_pred,
    color="red"
)

[<matplotlib.lines.Line2D at 0x7b9737107890>]
```



Lets the evaluations with metrics:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

MAE : 0.23150985393278373
MSE : 0.08417638361329656
RMSE: 0.2901316659954521
R2 Score: 0.7730984312051673
```

Lets print the Parameters our model found:

```
print("Intercept:", model.intercept_)  
print("Coefficients:", model.coef_)
```

```
Intercept: [-1.02700694]  
Coefficients: [[0.57425647]]
```